

AutoBench Service — Admin Guide

Last modified: 2026-09-29T04:20:05Z

Hand-maintained, unlike the generated `results/12run-*.md` files which stamp themselves. Bump the line above when you edit this guide.

How the AutoBench Service gets onto a cluster and stays trustworthy there: prerequisites, the environment that differs between platforms, the Helm chart, and the verification that actually proves the install. *Driving* the Service once it answers is a different document — [DEVELOPER_GUIDE.md](#).

- **Every host below is a placeholder on `example.com`** — this repo is public, so it names platforms rather than endpoints. Route *shapes* are real (`<service>--<namespace>.apps.<cluster>.example.com`), so substitute your own domain and the recipes work unchanged.
- **No secret value appears in this guide, and none should appear in your terminal.** Every script here reads credentials from the environment or from a `chmod 600` file and reports them only as a truncated SHA-256. If you find yourself about to `echo` one, don't.
- Two install paths exist and both are supported: the **Helm chart** at `deploy/helm/autobench` (§5) and the **raw manifests** under `deploy/` (§8). They render the same objects, and that equivalence is enforced by a check, not asserted.

In a hurry? §2.4 is the one command that tells you whether the cluster is ready, §5 is the install, §6 is the only verification that means anything.

Contents

- [1. What you install, and what you don't](#)
 - [The two cluster shapes](#)
- [2. Prerequisites](#)
 - [2.1 Rossoctl v0.8.0 or later](#)
 - [2.2 Workstation tooling](#)
 - [2.3 Cluster-side checklist](#)
 - [2.4 The preflight script](#)
- [3. Environment variables](#)
 - [3.1 Install-time — your shell only, never a values file](#)
 - [3.2 The Service pod](#)
 - [3.3 The workload pods — injected, not configured](#)
 - [3.4 KinD and OpenShift differ — and the differences fail silently](#)
 - [3.5 The LiteLLM gateway: two named profiles](#)
- [4. The instance-config Secret](#)
- [5. Installing with Helm](#)
 - [5.1 OpenShift](#)
 - [5.2 KinD](#)
 - [5.3 Adopting an install made from the raw manifests](#)

- 5.4 Upgrade, rollback, uninstall
- 6. Verifying the install
- 7. Symptoms that lie
- 8. Appendix: the raw-manifest path, and two KinD-only objects

1. What you install, and what you don't

The Service is a single stateless pod. It holds no benchmark data and no cluster credential: it authenticates to Rossoctl per request with its own ROPC identity, and Rossoctl performs every cluster operation server-side. Nothing here shells out to `kubectl` or `oc`.

object	name	created by	notes
Deployment	autobench-service	chart / deploy/deployment.yaml	1 replica, read-only root filesystem, arbitrary-UID safe
Service	autobench-service	chart / deploy/service.yaml	ClusterIP :8080, port named http
Route	autobench	chart (platform: openshift)	edge TLS; host generated as autobench-<namespace>.apps.<cluster>
HTTPRoute	autobench	chart (platform: kind)	attaches to the shared istio gateway
Secret	autobench-instances	you, out-of-band (§4)	ROPC credentials — never passes through Helm values

Everything else the Service depends on belongs to the platform and must exist **before** the install is useful:

- a Keycloak user (`benchmarker` by convention) with the `rossoctl-operator` realm role;
- `openai-secret` (`apikey`) and `hf-secret` (`hf-token`) in **every namespace you deploy benchmarks into** — `team1` unless you have changed the specs, since all 24 entries in `reference/run12_specs.json` name it;
- an OTEL collector that writes to an MLflow the Service can read (§3.4).

The two cluster shapes

shape	example	consequence
single-cluster — Service and agents together	KinD, ykt5	agents reach the collector over service DNS; no extra ingress
split — Service here, agents there	ykt3 hosts the Service, agents run on ykt2	the instance config carries <code>agent_endpoint_template/mcp_endpoint_template</code> , and the agents export telemetry back through an edge Route on <code>:443</code> whose target port resolves to the collector's 8335

The shape is not a chart value. It lives entirely in the instance config, so one chart installs both.

2. Prerequisites

2.1 Rossoctl v0.8.0 or later

This is the hard one, because falling short of it does not error. An older operator accepts the deploy request the Service sends and **silently drops fields it does not know** — `k8sResourceLimits` among them — so the benchmark comes up in a shape nobody configured and every subsequent measurement is quietly off.

Read the version off the **backend**. The operator subchart carries its own, lower, version line, and comparing that one rejects a perfectly current cluster:

```
oc -n rossoctl-system get deploy rossoctl-backend \
-o jsonpath='{.metadata.labels.app\.kubernetes\.io/version}{"\n"}' # 0.8.0-rc.2
```

A release-candidate suffix counts: `0.8.0-rc.2` satisfies " $\geq 0.8.0$ ". Both `reference/preflight.py` and `reference/ocp-service-bootstrap.sh` assert this before doing anything else.

2.2 Workstation tooling

tool	needed for	note
kubectl (or oc)	everything	oc only for Route conveniences; kubectl can do it all
helm ≥ 3.8	\$5	3.8 is the floor for the OCI/---kube-context behaviour used here
jq, curl	the bootstrap scripts	secrets move through jq via the <i>environment</i> , never argv
python3	preflight.py, helm-parity-check.py, gen_toc.py	kind-collector-mlflow.py also needs pyyaml
kind + a container engine	the KinD path only	the image is built locally and kind loaded

2.3 Cluster-side checklist

Each row is a failure that has actually been shipped, and the right-hand column is why you cannot find it by reading logs at the layer where it hurts.

prerequisite	where	if missing
Rossoctl $\geq$ v0.8.0	rossoctl-system	fields dropped from the deploy request, silently (§2.1)
agentruntimes + agentcards CRDs	cluster	/deploy fails at the operator
benchmarker user, ROPC enabled on the client	realm	token request 400s; on this realm a missing firstName/lastName yields Account is not fully set up
rossoctl-operator realm role on that user	realm	/deploy surfaces the operator's 403 as a 502
openai-secret / apikey	team1	empty or foreign key $\Rightarrow$ a 401 per completion, mid-run ; the leg finishes with zeroes rather than failing

prerequisite	where	if missing
hf-secret / hf-token (may be empty)	team1	MCP pod CreateContainerConfigError, the agent then crash-loops against it, and the <i>run</i> reports 424
OTEL collector, HTTP receiver on :8335	rossoctl-system	an agent pointed at 4318 hard-fails into CrashLoopBackOff
an MLflow the collector writes to and the Service can read	varies (§3.4)	the run passes with model: "unknown" and every token count 0

On the collector port: 8335 and only 8335. Every cluster's `otel-collector-config` declares receivers on 4317/4318, and reading that ConfigMap will tell you to use 4318. It is wrong — the Deployment moves the HTTP receiver with a command-line override, so 4318 never listens:

```
oc -n rossoctl-system get deploy otel-collector \
-o jsonpath='{.spec.template.spec.containers[0].command}'
# ["/otelcol-contrib","--config=/etc/otelcol-config/base.yaml",
#  "--set","receivers::otlp::protocols::http::endpoint=0.0.0.0:8335"]
```

The same principle applies to MLflow: read the target and the experiment id from the collector's own exporter block rather than choosing them. `traces_endpoint` and the `x-mlflow-experiment-id` header are what the *writer* uses, and an instance config that disagrees points the Service at an experiment nothing writes to.

2.4 The preflight script

`reference/preflight.py` checks every row of §2.3 against the live cluster. It is **read-only** — no writes, no deploys — and prints every credential as an 8-char SHA-256 prefix unconditionally, with no attempt to decide which values are sensitive.

```
python3 reference/preflight.py --platform kind --context kind-rossoctl
python3 reference/preflight.py --platform openshift --context <ykt5-ctx>
python3 reference/preflight.py --platform openshift --context <ykt3-ctx> \
--workload-context <ykt2-ctx> # split shape
python3 reference/preflight.py --json # machine-readable
```

Exit status is 0 when nothing FAILED; warnings do not fail the run. Twelve sections, in the order a request travels: tooling, cluster reachability, Rossoctl version and CRDs, namespaces, workload secrets, the collector, MLflow, ingress, the instance config, identity, an audit of any existing install, and the local chart.

```
Workload secrets (per team namespace)
ok    team1/openai-secret apikey - sha8 0239f193
ok    team1/hf-secret hf-token - empty value (fine)

OTEL collector (the write half of the telemetry chain)
ok    collector HTTP receiver on :8335 - read from the Deployment's command
ok    collector traces_endpoint - http://mlflow-reader.rossoctl-
system.svc.cluster.local:5000/v1/traces
ok    collector x-mlflow-experiment-id - 0

MLflow (the read half - this is what turns a run into a token report)
ok    MLflow traces API answers 200 - experiment 0, 1 trace(s) visible
ok    collector exports to mlflow-reader
...
47 ok, 0 warning(s), 0 failure(s) - ready to install
```

A clean preflight is necessary, not sufficient: only a 1-task leg with a non-zero token row proves the whole chain.

Three things about how it reads the cluster are worth knowing, because each was a false alarm first:

- **Ports and targets come from live objects**, never from a manifest or a ConfigMap — the collector's port from the Deployment's `command`, MLflow's identity from the exporter.
- **Route has no CRD**. It is served by the aggregated openshift-apiserver, so `get crd routes.route.openshift.io` reports it missing on a cluster that plainly serves Routes. The script asks `api-resources --api-group=route.openshift.io` instead.
- **A split shape is not a broken one**. With `agent_endpoint_template` in the instance config and no `--workload-context`, the workload checks are *skipped* rather than run against the wrong cluster — and a `workload_otel` endpoint on `:443` is resolved through the Route to the collector's port instead of being rejected.

Setting `KC_SERVICE_USERNAME` and `KC_SERVICE_PASSWORD` in the environment enables the two identity checks (an actual ROPC login, and the `rossoctl-operator` role). Without them those two are skipped, not failed.

3. Environment variables

There are three populations, and conflating them is how a key ends up in the wrong cluster. They are, in order: what you export **to run the installer**, what the **Service pod** reads, and what the operator injects into the **workload pods**.

3.1 Install-time — your shell only, never a values file

Nothing in this table is a Helm value, a ConfigMap, or a committed file. Each is read by a script, used once, and hashed if it is reported at all.

variable	used by	notes
KC_SERVICE_USERNAME	both bootstrap scripts, preflight.py	the ROPC login the Service uses against Rossoctl (benchmark)
KC_SERVICE_PASSWORD	both bootstrap scripts, preflight.py	required ; written into the instance file, never echoed
KC_SERVICE_CLIENT_SECRET	both bootstrap scripts	only if the Keycloak client is confidential
KC_USER_PASSWORD	kind-post-setup.sh	the password to <i>seed</i> ; falls back to <code>~/rossoctl-kind/benchmark.pass</code> (<code>KC_CRED_FILE</code>)
KC_ADMIN_PASSWORD	kind-post-setup.sh	optional — read from the in-cluster keycloak-initial-admin Secret when unset
LLM_PROFILE	both bootstrap scripts, kind-post-setup.sh, preflight.py	intranet internet — selects one of the two gateway variable sets (§3.5)
INTRANET_LLM_* / INTERNET_LLM_*	reference/llm-profiles.sh, read by all of the above	the profiles themselves: base, model, key file, optional bypass list (§3.5)

variable	used by	notes
BM_WORKLOAD_LLM_KEY	kind-post-setup.sh	the workload LLM key; falls back to the selected profile's key file, else ~/.rossoctl-kind/litellm.key (LLM_KEY_FILE)
S3_ACCESS_KEY_ID / S3_SECRET_ACCESS_KEY	ocp-service-bootstrap.sh	override --s3-from, which otherwise copies them in-process from an existing instance file
WORKLOAD_LLM_API_BASE, WORKLOAD_LLM_MODEL	both bootstrap scripts	set by LLM_PROFILE when you use one; an explicit export wins. The gateway has no default on purpose: this repo is public
MLFLOW_URL, MLFLOW_EXPERIMENT_ID, MLFLOW_WORKSPACE, MLFLOW_TOKEN_SECRET	ocp-service-bootstrap.sh	the id and workspace default to what the collector exports
WORKLOAD_OTEL_ENDPOINT, WORKLOAD_OTEL_INSECURE, WORKLOAD_AGENT_RUNNER	both bootstrap scripts	see §3.4 on the runner
IMAGE, CLUSTER, KUBE_CONTEXT, REALM, CLIENT, KC_HOST	kind-post-setup.sh	plain overrides, no secrets

Two conventions here are deliberate and both cost real time when broken:

The LLM key is `BM_WORKLOAD_LLM_KEY`, not `OPENAI_API_KEY`. That name is commonly exported in a developer's shell profile for an unrelated provider, and the script writes whatever it finds into cluster Secrets. A namespaced name cannot be inherited by accident. (Inside the pod the value still arrives as `OPENAI_API_KEY` — the registry maps it from the Secret's `apikey`.)

Credential files are refused unless they are `600` or `400`, and they are meant to outlive the cluster: neither the `benchmarker` password nor the LLM key changes across a rebuild.

```
umask 077; mkdir -p ~/.rossoctl-kind
printf '%s' '<benchmarker password>' > ~/.rossoctl-kind/benchmarker.pass
printf '%s' '<llm key>' > ~/.rossoctl-kind/litellm.key
```

3.2 The Service pod

The container reads exactly two environment variables from the Deployment, and both are set by the chart:

variable	value	why
SERVICE_INSTANCES_DIR	/etc/service/instances	where the autobench-instances Secret is mounted read-only
SERVICE_PORT	8080	read by the entrypoint (uvicorn), not by the settings model

Everything else is a `SERVICE_`-prefixed setting with a working default, overridable through `extraEnv` in the values file. These are the ones worth knowing:

setting	default	when you would change it
SERVICE_LOG_LEVEL	INFO	DEBUG while diagnosing an auth or deploy failure

setting	default	when you would change it
SERVICE_HTTP_TIMEOUT_SECONDS	30	a slow Rossoct! API
SERVICE_JWKS_CACHE_SECONDS	300	rarely
SERVICE_EXPORT_SETTLE_MAX_SECONDS	20	a fast run can be exported <i>before</i> its child spans land, yielding model: "unknown" and zero tokens; the Service re-reads until the record set stops growing, and this is that budget
SERVICE_EXPORT_SETTLE_INTERVAL_SECONDS	2	with the above
SERVICE_SPAN_REPORT_MAX_ROWS	200000	only to cap a pathological trace — the export is built in memory

```
# values fragment
extraEnv:
  - name: SERVICE_LOG_LEVEL
    value: DEBUG
```

The settings model uses Pydantic's `extra="ignore"`, so a **misspelled `SERVICE_*` name is accepted and discarded without a word**. If a setting seems not to take effect, suspect the spelling before the code.

3.3 The workload pods — injected, not configured

You do not set these. The Service derives them per deploy from the benchmark definition (`src/autobench/benchmarks/registry.py`) and the instance config, which is why an agent's environment cannot drift from the run that measured it. They are listed so you can recognise them in a pod spec:

variable	source	note
OPENAI_API_KEY	secretKeyRef → openai-secret / apikey	never a literal value in the pod spec
HF_TOKEN	secretKeyRef → hf-secret / hf-token	gsm8k only; presence is the requirement, not content
OPENAI_API_BASE, LLM_API_BASE	instance workload_llm.api_base	any inherited values are dropped and re-injected
EXGENTIC_DEFAULT_RUNNER	instance workload_agent_runner	§3.4
EXGENTIC_OTEL_ENABLED	true when workload_otel.enabled	
OTEL_EXPORTER_OTLP_ENDPOINT / _PROTOCOL / _INSECURE	instance workload_otel	the endpoint is the :8335 one

3.4 KinD and OpenShift differ — and the differences fail silently

The Service's own objects are nearly identical across platforms. What differs is everything it points *at*.

	KinD	OpenShift	if you get it wrong
LLM gateway	usually the internal gateway	usually the external one — but this follows the <i>network</i> , not the platform (§3.5)	the two keep separate key tables, so a key moved across does not fail closed — it 401s per completion, mid-run
openai-secret	a key from the internal gateway	a key from the external one	same
MLflow read path	mlflow-reader (§8), no auth	the cluster's own MLflow on :8443 with a ServiceAccount bearer and insecure_tls	a refused read is invisible: the run passes, every token count is 0
MLflow experiment	0	1, workspace team1 on ykt5	identical signature to the above
Keycloak dial	the backchannel service DNS — iss is unreachable in-cluster, since *.localtest.me resolves to pod loopback and there is no CoreDNS rewrite	the iss Route itself	JWKS and ROPC both fail at startup
collector endpoint	http://otel-collector.rossocctl-system.svc.cluster.local:8335	the same on a single-cluster install; an edge Route on :443 when the agents live elsewhere	unreachable collector ⇒ agent CrashLoopBackOff, surfacing as a 424 on the run
pod security context	UID/GID/fsGroup pinned to 10001/0/10001	runAsNonRoot + seccomp only	pinning 10001 can be rejected when it falls outside the project's allocated UID range
ingress	HTTPRoute on the shared gateway	Route, edge TLS	—

workload_agent_runner **deserves its own warning**. The correct value tracks an agent image pinned to :latest, and it has flipped between **direct** and **service** across rebuilds of that tag. With the current image, **service** is what emits agent spans; **direct** yields a clean run with **zero-token rows**, which is exactly what a broken MLflow looks like. Do not infer it — detect it, by looking for a non-zero token row in report.ndjson (§6).

3.5 The LiteLLM gateway: two named profiles

The gateway is the one piece of configuration that is neither discoverable from the cluster nor shared between clusters, so it gets its own section — and it is configured as a **named profile** rather than as a set of per-cluster values, because the failure it causes is silent.

The deciding question is **where the cluster's pods sit on the network**, not which Kubernetes it runs. An OpenShift cluster deployed on the organisation's intranet uses the same internal gateway a local KinD cluster on the VPN does; a KinD cluster on a laptop off the VPN uses the external one. **platform** and **llmProfile** are therefore independent, and nothing derives one from the other.

	intranet	internet
which gateway	the internal LiteLLM (...vpc-int...)	the external LiteLLM (...vpc...)

	intranet	internet
who uses it	any cluster whose pods are inside the org network — a personal KinD on the VPN, or an intranet OpenShift cluster	a cluster reached from outside it
why not the other	the external host is not routed on a split-tunnel VPN — connect times out	the internal host is in a routed private range, reachable only from inside
model catalogue	ids from <i>that</i> gateway	not the same ids
key table	its own	its own — this is the whole problem

The two variable sets

Each profile is one set of environment variables, distinguished only by prefix, defined in `reference/llm-profiles.sh`:

intranet	internet	what it is
INTRANET_LLM_API_BASE	INTERNET_LLM_API_BASE	the gateway origin — scheme + host, no path
INTRANET_LLM_MODEL	INTERNET_LLM_MODEL	a model id from that gateway's catalogue
INTRANET_LLM_KEY_FILE	INTERNET_LLM_KEY_FILE	chmod 600 file holding that gateway's key (default <code>~/.rossoctl-llm/<profile>.key</code>)
INTRANET_LLM_NO_PROXY	INTERNET_LLM_NO_PROXY	optional explicit egress-proxy bypass list; otherwise built for you

`LLM_PROFILE=intranet|internet` (or `--llm-profile`) selects one, and the scripts copy it into the canonical `WORKLOAD_LLM_API_BASE` / `WORKLOAD_LLM_MODEL` / `LLM_KEY_FILE` they already read. An explicitly exported `WORKLOAD_LLM_*` still wins, so nothing that worked before behaves differently.

Bases and model ids are not secrets, so keep them in a file instead of re-exporting per shell. The **keys stay out of it**, in their own per-profile files:

```
mkdir -p ~/.rossoctl-llm
cat > ~/.rossoctl-llm/profiles.env <<'EOF'
INTRANET_LLM_API_BASE=https://<internal gateway host>
INTRANET_LLM_MODEL=openai/aws/claude-haiku-4-5
INTERNET_LLM_API_BASE=https://<external gateway host>
INTERNET_LLM_MODEL=openai/Azure/gpt-5-mini-2025-08-07
EOF
umask 077
printf '%s' '<internal gateway key>' > ~/.rossoctl-llm/intranet.key
printf '%s' '<external gateway key>' > ~/.rossoctl-llm/internet.key
```

Per-profile key files are the point: the two tables are separate, so one file holding "the" key is exactly how a key gets used against the gateway that never issued it.

Selecting a profile for a deployment

The profile is declared **once**, as `llmProfile` in the chart values file, and everything else reads it from there:

```
# deploy/helm/values-kind.yaml          # deploy/helm/values-ykt5.yaml
```

```
platform: kind                                # platform: openshift
llmProfile: intranet                         # llmProfile: internet
```

```
reference/ocp-service-bootstrap.sh --llm-profile internet --cluster ykt5 --context <ctx> ...
LLM_PROFILE=intranet reference/kind-post-setup.sh
python3 reference/preflight.py --values deploy/helm/values-kind.yaml --context kind-rossctl
```

The chart **does not** render the gateway into the Service pod, and cannot: the Service reads the gateway from `workload_llm.api_base` in the instance-config Secret (§4) — not from its own environment — and injects `OPENAI_API_BASE` / `LLM_API_BASE` onto each workload pod at deploy time (§3.3). So `llmProfile` is a **declaration**, and it does two things that matter:

- the bootstrap scripts take the profile from it, so the instance file they write cannot disagree with the release installed beside it;
- `preflight.py` **FAILS** when the live instance file's `api_base` is not that profile's base:

```
FAIL keycloak.localtest.me_8080.json: LLM gateway matches the internet profile – instance
points
  at <internal host>, the internet profile declares <external host> – separate key tables,
so
  this 401s per completion mid-run rather than failing at deploy
```

That is the entire point of naming the profiles: it converts a mid-run 401 storm, whose only visible symptom is a leg that finishes with zeroes, into a pre-install error. `helm` rejects a mistyped profile for the same reason — `llmProfile: intranett` fails the render rather than being quietly ignored downstream.

Four details that have cost time

- **The key never travels in the instance file.** It lives only in `openai-secret` / `apikey` in each workload namespace, and reaches the pod as `OPENAI_API_KEY` through a `secretKeyRef`.
- **The proxy-bypass list is part of the gateway's configuration, not a separate concern.** Where the cluster injects an egress proxy, an in-network gateway base must be excluded from it or every completion leaves through a proxy that cannot reach it. `workload_llm.no_proxy` (and `disable_proxy: true`, which injects an empty `HTTP_PROXY` / `http_proxy`) are rendered onto the **agent** pod only. Both bootstrap scripts now build the list — loopback, the gateway host, the collector, and Keycloak's in-cluster service — where before only the OpenShift one did, so a KinD bootstrap without `--copy-from` silently lost it.
- **WORKLOAD_LLM_BASE was never a second setting.** It is an accepted alias for `WORKLOAD_LLM_API_BASE` in the KinD script, kept only so existing invocations keep working; the canonical name matches the instance-file field (`workload_llm.api_base`) on both platforms.
- **Do not verify a gateway with curl.** LiteLLM strips the `openai/` prefix, so the configured model name 403s by hand, and reasoning models reject `max_tokens` with a 400 (`max_completion_tokens` is the accepted field). The only trustworthy probe is a 1-task `gsm8k` leg (§6).

4. The instance-config Secret

One JSON file per issuer, keyed by `iss`, mounted read-only at `/etc/service/instances`. **No install path creates it:** it carries the ROPC service credential and the S3 keys, so it is generated out-of-band and must

never pass through Helm values or a values file.

One script per platform, so the file has reproducible provenance instead of being hand-assembled:

```
# OpenShift – runs eleven prechecks before it writes anything
KC_SERVICE_USERNAME=benchmarker KC_SERVICE_PASSWORD=... \
reference/ocp-service-bootstrap.sh --cluster ykt5 --context <ctx> \
  --apps-domain apps.ykt5.example.com \
  --llm-api-base https://<external-gateway>/v1 \
  --out-dir instances/
# KinD
KC_SERVICE_USERNAME=benchmarker KC_SERVICE_PASSWORD=... \
reference/kind-service-bootstrap.sh --context kind-rossoctl \
  --copy-from instances/keycloak.localtest.me_8080.json \
  --out-dir instances/
```

Then, on either platform:

```
kubectl -n rossoctl-system create secret generic autobench-instances \
  --from-file=<encoded-iss-host>.json=instances/<encoded-iss-host>.json" \
  --dry-run=client -o yaml | kubectl apply -f -
kubectl -n rossoctl-system rollout restart deploy/autobench-service
```

The filename encodes the `iss host` with `:` rewritten to `_`; the exact `iss` inside the file is the source of truth. The Service reads instance files at startup, hence the restart. Note that `PUT /config` is in-memory only — a correction that must survive a restart goes in the Secret.

Four properties of these scripts are load-bearing:

- **Values reach `jq` through the environment, not `--arg`**. `argv` is world-readable (`ps`, `/proc/<pid>/cmdline`); a process's environment is not.
- **The file is created `600` by `umask`**, not `chmod` ed afterwards, so there is no window in which credentials sit world-readable on disk.
- **`--copy-from` / `--s3-from` carry `s3` and `workload_llm` over** from an existing file. Those two are environment facts a script cannot discover, and a cluster rebuild should reproduce them rather than lose them — losing them is silent, and costs a run's artifacts.
- **A no-auth MLflow still needs a `bearer_token`**, which is not a contradiction: the Service mints a token *before* it reads, and with neither a bearer nor client-credentials the token helper raises, the route catches it, and the run fails soft into an empty token report. For the KinD reader the scripts emit an ignored placeholder.

5. Installing with Helm

```
deploy/helm/autobench/
  Chart.yaml          version = chart version; appVersion = the default image tag
  values.yaml         the two platform shapes, documented inline
  templates/          deployment, service, route, httproute, _helpers.tpl, NOTES.txt
deploy/helm/values-ykt5.yaml  an OpenShift cluster's overrides
deploy/helm/values-kind.yaml  KinD's overrides
```

`platform` selects the shape and is the only value most installs need to think about. It drives the pod security context and which ingress object is rendered (§3.4). The chart **references but never creates** `autobench-`

instances.

Parity with the raw manifests is the chart's correctness gate, and it has teeth. Run this after any template change:

```
helm lint deploy/helm/autobench
python3 reference/helm-parity-check.py      # 6 checks; "Chart and manifests agree."
```

It renders both platform shapes and diffs them against

`deploy/{deployment,service,kind/httproute}.yaml` and against the OpenShift patched render. The standard `app.kubernetes.io/*` labels are deliberately **absent**: the sole label and selector is `app: autobench-service`, because that is what the live Deployments select on, and a Deployment's selector is immutable. If the check fails, the chart is wrong — not the manifests.

5.1 OpenShift

```
python3 reference/preflight.py --platform openshift --context <ctx>      # 0 failures first
helm upgrade --install autobench deploy/helm/autobench \
  -n rossoctl-system --kube-context <ctx> -f deploy/helm/values-ykt5.yaml
oc -n rossoctl-system rollout status deploy/autobench-service
HOST=$(oc -n rossoctl-system get route autobench -o jsonpath='{.spec.host}')
curl -fsS "https://$HOST/healthz"                                     # {"status":"ok"}
```

Leave `route.host` empty unless you need a specific name — the router generates `autobench-
<namespace>.apps.<cluster>`, which is the shape every recipe in the developer guide assumes.

5.2 KinD

DEV/TEST only. On a **freshly rebuilt** cluster do not start here: `reference/kind-post-setup.sh` builds and `kind load` s the image, re-seeds the `benchmarker` user, ensures the team secrets, generates the instance config and creates the Secret. The chart is the last step of that, and alone it installs a pod with nothing to authenticate as.

```
IMAGE=ghcr.io/rossoctl/autobench:v1.29 reference/kind-post-setup.sh      # first time / after a
rebuild
python3 reference/preflight.py --platform kind --context kind-rossoctl
helm upgrade --install autobench deploy/helm/autobench \
  -n rossoctl-system --kube-context kind-rossoctl -f deploy/helm/values-kind.yaml
curl -fsS http://autobench.localtest.me:8080/healthz                    # {"status":"ok"}
```

8080 there is the **host** port KinD publishes the istio gateway on; it is not this Service's port, and the two matching is a coincidence. Note also that `kind load docker-image` bypasses the registry, so a KinD pod's digest will not match the published one even at the same tag — compare source, not digests, there.

5.3 Adopting an install made from the raw manifests

A chart cannot replace an existing Deployment — the selector is immutable — but it can take ownership of one. Both clusters that predate the chart were adopted this way rather than reinstalled, which is why neither had an outage: **the running pod is not recreated.**

```
for obj in deploy/autobench-service svc/autobench-service httproute/autobench; do # or
route/autobench
```

```
kubectl -n rossoctl-system annotate "$obj" \
  meta.helm.sh/release-name=autobench meta.helm.sh/release-namespace=rossoctl-system --
  overwrite
kubectl -n rossoctl-system label "$obj" app.kubernetes.io/managed-by=Helm --overwrite
done
helm upgrade --install autobench deploy/helm/autobench -n rossoctl-system -f <values>
```

Without the annotations Helm refuses the install with "invalid ownership metadata". Afterwards, confirm the pod's `restartCount` and `creationTimestamp` are unchanged — adoption should be a metadata-only operation.

Read release ownership from the Secrets, not from `helm list`. A `helm list --kube-context` comparison across two clusters has returned identical output for different clusters here; the release objects cannot lie about where they live:

```
kubectl -n rossoctl-system get secret -l owner=helm,name=autobench --context <ctx>
# sh.helm.release.v1.autobench.v1
```

5.4 Upgrade, rollback, uninstall

```
helm upgrade autobench deploy/helm/autobench -n rossoctl-system -f <values> \
  --set image.tag=v1.30
helm history autobench -n rossoctl-system
helm rollback autobench 1 -n rossoctl-system
helm uninstall autobench -n rossoctl-system # leaves autobench-instances behind
```

`helm uninstall` deletes only what the chart owns, so the instance Secret survives — which is what you want, since regenerating it means re-reading credentials. The image tag is immutable, so `imagePullPolicy: IfNotPresent` is correct and an upgrade means changing `image.tag`, never restarting to pick up a rebuild.

6. Verifying the install

`/healthz` proves the pod runs. It proves nothing about the chain the pod exists to drive, and every link in that chain has a failure mode that still returns 200. Work outward:

```
TOKEN=... # ROPC; see DEVELOPER_GUIDE.md §3
curl -fsS "$BASE/healthz"
curl -fsS -H "Authorization: Bearer $TOKEN" "$BASE/benchmarks" | jq '.items[].name'
curl -fsS -H "Authorization: Bearer $TOKEN" "$BASE/benchmarks/gsm8k" | jq
  '.agents[0].container_image'
```

`GET /benchmarks` returns `items`, not `benchmarks` — that has broken scripts twice — and it only summarises each benchmark (`name`, `mcp_image`, `agents`, `default_model`). An agent's `container_image`, `extra_env` or `model_override` is visible only through `GET /benchmarks/<name>`, so confirming a catalog change against the list endpoint alone can pass while the thing you changed is still the old value.

Then the only check that means anything: **a 1-task gsm8k leg whose report carries a non-zero token row.**

```
uv run autobench-cli --base "$BASE" run gsm8k --max-tasks 1 # see DEVELOPER_GUIDE.md §7.1
```

what you look at	expected	what it proves
run status	completed, pass rate 1.0	Service → Rossoctl → operator → agent → MCP → evaluator
tokens on the task row	non-zero (~320 in / 87 out)	agent → collector → MLflow → Service, the whole telemetry chain
model on the row	the gateway's model id, not "unknown"	the Service read the experiment the collector wrote to
pod imageID digest	matches the tag you installed (OpenShift)	the pod is not serving a stale : latest

A zero-token row is the single most informative failure in this system, because it is the shared signature of four unrelated causes: a refused MLflow read, an experiment-id mismatch, a collector that cannot be reached, and `workload_agent_runner` set to the value the current agent image does not emit spans for. Preflight separates the first three; only a run separates the fourth.

7. Symptoms that lie

symptom	actual cause	how to confirm
run passes, every token count 0, model: "unknown"	refused MLflow read, wrong experiment id, unreachable collector, or the wrong <code>workload_agent_runner</code>	\$2.4, then \$6
/deploy returns 502	the operator returned 403 — the service user lacks the rossoctl-operator realm role	the role mapping in the realm
/deploy returns 424 , agent CrashLoopBackOff	collector endpoint on 4318, or hf-secret missing so the MCP pod never started	the collector's command; get secret hf-secret
token request 400 Account is not fully set up	the realm requires firstName/lastName for ROPC	the user's profile
tasks error on the first completion, run finishes with zeroes	openai-secret empty, or holding the other gateway's key	preflight.py prints both namespaces' sha8; compare them
Model endpoint ... is unreachable on every task	the agent pod is running dev145 , whatever : latest points at	the wording: dev146 says did not respond ... after N attempt(s). Compare the pod's imageID digest, never the tag
a config change appears to do nothing	InstanceConfig and the SERVICE_* settings both use extra="ignore" — an unknown field is dropped silently	read the value back; adding one is never self-verifying
the new api_base is ignored	a stale Deployment from an earlier bring-up is still serving the old one	list Deployments in the team namespaces by age
a pod serves old code at the right tag	: latest agent/MCP images: a <i>running</i> pod never re-pulls	tear down before deploying; compare .status.containerStatuses[].imageID

symptom	actual cause	how to confirm
mlflow-reader restarts ~30 s after boot with no error in the log	OOMKilled — one MLflow 3.x worker idles at ~2.3 GiB, so a 2Gi limit is below the floor	the container's lastState, not its log
get crd routes.route.openshift.io says Routes are missing	Routes come from the aggregated apiserver and have no CRD	kubectl api-resources --api-group=route.openshift.io
/healthz 503, deploy 500 httpx.ReadError, agent CrashLoop — all at once, on KinD	a ztunnel restart un-enrolled older pods from the ambient mesh	recycle every pod older than ztunnel's Ready condition, in all namespaces

8. Appendix: the raw-manifest path, and two KinD-only objects

The manifests under `deploy/` remain supported and are what the chart is checked against:

```
kubectl apply -f deploy/service.yaml -f deploy/deployment.yaml
oc patch deploy autobench-service --patch-file deploy/openshift/deployment-patch.yaml #
OpenShift
kubectl apply -f deploy/kind/httproute.yaml # KinD
```

On OpenShift the patch must go on **before** the rollout, not after: it is what removes the pinned UID that restricted-v2 would otherwise reject.

Two objects live under `deploy/kind/` and are in neither the chart nor the OpenShift path, because neither has an OpenShift counterpart.

`mlflow-reader.yaml` — stock MLflow, no auth, serving the *same* postgres as the writer. It exists because the MLflow `rossocctl-deps` installs runs `mlflow-oidc-auth`, which rejects both the Service's read and the collector's write; the run then publishes zeros on every task, which reads like missing telemetry rather than a refused read. It is a "reader" by convention only. Two of its settings are measured rather than chosen: `--workers 1` (MLflow 3.x defaults to four) and a **4Gi** limit. Gate on a 200 from `/api/2.0/mlflow/traces` rather than on the pod going Ready — two pip installs run at container start — and pass `experiment_ids`, since that endpoint answers 400 without it. Probe from **inside** the pod (MLflow 3.x rejects the API server's service proxy as a DNS-rebinding attempt) with `python`, not `curl`, which the image does not ship. `reference/kind-post-setup.sh` does all of this, including pointing the collector at the reader; by hand it is:

```
python3 reference/kind-collector-mlflow.py # patch + restart; idempotent
python3 reference/kind-collector-mlflow.py --check # report only, exit 1 if unpatched
```

`ibac-judge.yaml` — the upstream proxy the AuthBridge `ibac` plugin calls. The plugin's ConfigMap is cleartext, so the key cannot live there; this holds it in a Secret and adds the `Authorization` header outbound, leaving the plugin pointed at a credential-free in-cluster URL. Without it the plugin is **inert**: it admits every call and the run looks like a clean pass, which is why enforcement is proven by *counting judge calls* and never by reading a pass rate. `UPSTREAM_BASE` in the committed copy is an `example.com` placeholder — substitute the real gateway at apply time, and `rollout restart` after the key lands, since the proxy reads it once at pod start.